Guide du Batch Spring Boot

Reprise de données multi-tables

Architecture, mise en œuvre et guide pratique pour concevoir un batch de reprise de données où une table source s'éclate en plusieurs tables cibles.

Document technique
Avril 2026

Table des matières

1. Introduction et concepts

- 1.1 Qu'est-ce qu'une reprise de données
- 1.2 Pourquoi Spring Batch
- 1.3 Concepts clés : Job, Step, Chunk

2. Architecture globale du batch

- 2.1 Le pattern staging
- 2.2 Enchaînement des steps
- 2.3 Choix JPA vs JDBC

3. Architecture du projet

- 3.1 Structure des dossiers
- 3.2 Rôle de chaque package
- 3.3 Configuration multi-datasources

4. Problématiques clés

- 4.1 Gestion des clés étrangères
- 4.2 Éviter les doublons
- 4.3 Idempotence et rejeu

5. Ajouter un nouveau batch (pas à pas)

6. Bonnes pratiques et checklist

1. Introduction et concepts

1.1 Qu'est-ce qu'une reprise de données

Une **reprise de données** (RDD) est une opération qui consiste à migrer, transformer ou réinjecter des données d'un système source vers un système cible. Elle intervient typiquement dans les cas suivants :

- Migration d'application (ancien SI vers nouveau)
- Changement de modèle de données (refonte du schéma BDD)
- Correction massive de données existantes
- Initialisation d'un nouveau système avec des données historiques

C'est un traitement **ponctuel ou périodique**, généralement sur de gros volumes, qu'on automatise via un batch pour garantir reproductibilité, traçabilité et performance.

> **Attention à l'acronyme :** RDD peut aussi désigner *Resilient Distributed Dataset*, la structure de base d'Apache Spark (Big Data). Dans ce document, RDD = Reprise De Données.

1.2 Pourquoi Spring Batch

Spring Batch est un framework conçu spécifiquement pour les traitements batch à grande échelle. Il fournit nativement :

- **Gestion transactionnelle par chunks** : on traite par paquets, avec commit à chaque paquet
- **Reprise sur incident** : un job qui plante peut redémarrer où il s'est arrêté
- **Tolérance aux erreurs** : skip, retry, listeners pour tracer les rejets
- **Métadonnées persistées** : tables `BATCH_JOB_EXECUTION`, etc.
- **Parallélisation** : partitioning, multi-threading
- **Readers et Writers standards** : JDBC, JPA, fichiers plats, XML, JSON

1.3 Concepts clés

Concept	Rôle
Job	Le traitement complet (ex : "reprise clients"). Composé d'un ou plusieurs steps.
Step	Une étape du job. Peut être de type Chunk (lecture/traitement/écriture) ou Tasklet (action unique).
ItemReader	Lit les données unitairement depuis une source (BDD, fichier...).
ItemProcessor	Transforme un item (validation, mapping, enrichissement). Optionnel.
ItemWriter	Écrit les items par paquets dans une cible.
Chunk	Le paquet d'items. Ex : chunk-size = 500 → commit toutes les 500 lignes.
Tasklet	Une action unique non itérative (ex : exécuter un UPDATE SQL massif).
JobRepository	Persiste l'état d'exécution des jobs et steps.

2. Architecture globale du batch

2.1 Le pattern staging

Pour une RDD volumineuse où une table source s'éclate en plusieurs tables cibles, l'approche recommandée est le **pattern staging** : on intercale une couche de tables intermédiaires entre la source et les cibles.

Avantages du staging

- **Figer les données source** à un instant T : si le batch dure 4h et que la source évolue, on est protégé
- **Découpler extraction et chargement** : on peut relancer l'un sans l'autre
- **Tracer l'état de chaque ligne** : statut NEW / OK / KO / SKIP, motif de rejet
- **Dédoublonner et valider** via des requêtes SQL massives (rapides)
- **Rejouer facilement** en cas d'incident, sans toucher à la source

2.2 Enchaînement des steps

Le job se décompose typiquement en 5 phases :

Phase	Step(s)	Technique recommandée
1. Extraction	Source → Staging	JdbcCursorItemReader + JPA Writer
2. Validation	Marquage KO des lignes invalides	Tasklet SQL (UPDATE massif)
3. Déduplication	Marquage SKIP des doublons	Tasklet SQL
4. Chargement	Staging → tables cibles (1 step par table)	JpaItemReader/Writer
5. Réconciliation	Comptage source vs cible, rapport	Tasklet

> **Ordre d'insertion :** les steps de chargement doivent respecter l'ordre *parent avant enfants* pour préserver les contraintes de clés étrangères naturellement. Exemple : CLIENT avant COMMANDE, COMMANDE avant LIGNE_COMMANDE.

2.3 Choix JPA vs JDBC

Le choix dépend du volume et de la complexité métier :

Volume	Approche recommandée	Justification
< 100 000 lignes	JPA partout	Confort, cascades automatiques, perfs suffisantes
100 000 à quelques millions	JPA avec batch_size configuré	Bon compromis avec `hibernate.jdbc.batch_size=500`
Dizaines de millions	Hybride : JPA pour logique métier, JDBC pour volumes	JDBC brut pour extraction/chargement massif
Centaines de millions	Bulk natif (COPY, SQL*Loader)	Seul moyen de tenir les délais

Configuration JPA pour batch

```
spring:
  jpa:
    properties:
      hibernate:
        jdbc.batch_size: 500
        order_inserts: true
        order_updates: true
        generate_statistics: false
```

3. Architecture du projet

3.1 Structure des dossiers

L'organisation suit le principe : **par domaine métier d'abord, par couche technique ensuite**.

```
reprise-donnees-client/
├── pom.xml
├── README.md
├── docs/                              ← documentation fonctionnelle
│   ├── mapping-source-cible.md        ← LE document clé en RDD
│   ├── regles-validation.md
│   └── runbook-exploitation.md
│
├── scripts/sql/
│   ├── ddl/                           ← création des tables
│   ├── pre-batch/                     ← désactivation index
│   └── post-batch/                    ← réconciliation, rapports
│
└── src/main/java/fr/montreprise/reprise/
    ├── RepriseDonneesApplication.java ← point d'entrée
    │
    ├── config/                        ← config transverse
    │   ├── BatchConfig.java
    │   ├── DataSourceConfig.java      ← source, cible, batch
    │   ├── JpaConfig.java
    │   └── AppProperties.java
    │
    ├── job/                           ← définition des JOBS
    │   ├── RepriseClientJobConfig.java
    │   └── listener/
    │       └── JobExecutionListener.java
    │
    ├── step/                          ← STEPS par phase
    │   ├── extraction/
    │   ├── validation/
    │   ├── deduplication/
    │   ├── chargement/
    │   │   ├── client/
    │   │   ├── adresse/
    │   │   ├── contact/
    │   │   └── compte/
    │   ├── reconciliation/
    │   └── listener/
    │       └── SkipListener.java
    │
    ├── domain/                        ← MODÈLE MÉTIER par bounded context
    │   ├── legacy/                    ← entités SOURCE (lecture seule)
    │   ├── staging/                   ← tables de staging
    │   ├── cible/                     ← entités CIBLE (écriture)
    │   │   ├── client/
    │   │   ├── adresse/
    │   │   └── contact/
    │   └── rejet/                     ← traçabilité
    │
    ├── mapper/                        ← transformation source↔cible
    │   ├── ClientMapper.java
    │   └── AdresseMapper.java
    │
    ├── service/                       ← logique métier réutilisable
    │   ├── validation/
    │   ├── enrichissement/
    │   └── rejet/
    │
    └── exception/
        ├── RepriseException.java
        └── DonneeInvalideException.java
```

3.2 Rôle de chaque package

Package	Rôle
`config/`	Configuration transverse : datasources, EntityManagers, JobRepository
`job/`	Assemblage des steps dans un ordre précis. Point d'entrée pour comprendre le flow.
`step/`	Définitions détaillées de chaque step avec Reader/Processor/Writer. Organisé par phase.
`domain/legacy/`	Entités qui mappent l'ancienne BDD. Lecture seule, `@Immutable`.
`domain/staging/`	Tables techniques du batch. Durée de vie = temps du batch. Portent les colonnes de pilotage (statut, motif_rejet).
`domain/cible/`	Entités de la nouvelle BDD. Celles qui vont rester en production.
`domain/rejet/`	Traçabilité des lignes rejetées. Conservé après le batch pour audit.
`mapper/`	Logique de transformation source→cible. Testable unitairement sans Spring Batch.
`service/`	Logique métier réutilisable entre plusieurs processors : validation, enrichissement, référentiels.
`exception/`	Exceptions métier et techniques spécifiques à la reprise.

3.3 Configuration multi-datasources

Une RDD a toujours au minimum **deux datasources** (source et cible), voire trois avec la base de métadonnées Spring Batch séparée.

```
@Configuration
public class DataSourceConfig {

    @Bean
    @ConfigurationProperties("app.datasource.source")
    public DataSource sourceDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean @Primary
    @ConfigurationProperties("app.datasource.cible")
    public DataSource cibleDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean @BatchDataSource
    @ConfigurationProperties("app.datasource.batch")
    public DataSource batchDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

4. Problématiques clés

4.1 Gestion des clés étrangères

Les FK entre tables cibles posent la question : faut-il les désactiver pendant le chargement ? **La réponse est : non, dans 90% des cas.**

Règle générale

Si tu respectes l'ordre **parent → enfants** dans tes steps, les FK ne te gênent pas : quand tu insères une adresse, le client référencé existe déjà. C'est la raison d'être du découpage par steps.

Quand désactiver les FK peut se justifier

- Volumes très importants (dizaines de millions de lignes) : gain de perf de 20-40%
- Ordre d'insertion impossible à garantir (cas rare)
- Données circulaires (A référence B qui référence A)

> **Attention :** désactiver les FK fait perdre l'intégrité pendant le batch. En cas de plantage, on peut se retrouver avec des orphelins. Toujours *réactiver avec validation* à la fin (`ENABLE VALIDATE CONSTRAINT` sur Oracle, `ADD CONSTRAINT` sur PostgreSQL) pour détecter les incohérences.

Alternatives plus efficaces

Avant de désactiver les FK, considérer :

- Désactiver les **index secondaires** (pas les PK/FK) pendant le chargement
- Tables en mode **UNLOGGED** (PostgreSQL) ou **NOLOGGING** (Oracle)
- Augmenter le **chunk size** à 1000-5000 si la mémoire suit
- **Paralléliser** les steps non dépendants
- Utiliser du **bulk insert natif** (`COPY` , `SQL*Loader`)

4.2 Éviter les doublons

Plusieurs stratégies complémentaires à combiner selon le cas :

Stratégie 1 : la clé métier stable

Chaque entité cible doit avoir une **clé métier** issue de la source (typiquement `id_externe`) avec une contrainte UNIQUE.

```
CREATE TABLE client (
    id BIGSERIAL PRIMARY KEY,
    id_externe BIGINT NOT NULL UNIQUE,  -- ← contrainte UNIQUE
    nom VARCHAR(100)
);
```

Stratégie 2 : UPSERT (recommandée)

Rend le batch **idempotent** : le rejouer ne crée pas de doublons. C'est la solution la plus propre et la plus rapide.

```
-- PostgreSQL
INSERT INTO client (id_externe, nom, prenom, email)
VALUES (:idExterne, :nom, :prenom, :email)
ON CONFLICT (id_externe) DO UPDATE SET
    nom = EXCLUDED.nom,
    prenom = EXCLUDED.prenom,
    email = EXCLUDED.email;

-- Oracle
MERGE INTO client c
USING (SELECT :idExterne AS id_externe FROM dual) src
ON (c.id_externe = src.id_externe)
WHEN MATCHED THEN UPDATE SET nom = :nom
WHEN NOT MATCHED THEN INSERT (id_externe, nom) VALUES (:idExterne, :nom);
```

Stratégie 3 : pré-check en mémoire

Viable jusqu'à quelques millions de clés :

```
@Bean
@StepScope
public ItemProcessor<ClientLegacy, Client> clientProcessor() {
    Set<Long> existants = new HashSet<>(
        jdbcTemplate.queryForList("SELECT id_externe FROM client", Long.class)
    );
    return legacy -> existants.contains(legacy.getId())
        ? null  // skip, déjà migré
        : mapToClient(legacy);
}
```

Stratégie 4 : dédoublonnage dans le staging

Utile quand la source contient déjà des doublons (même personne en double...) :

```
UPDATE stg_client_legacy
SET statut_traitement = 'SKIP_DOUBLON',
    motif_rejet = 'Email en doublon, ligne plus récente conservée'
WHERE id NOT IN (
    SELECT MAX(id) FROM stg_client_legacy
    GROUP BY LOWER(TRIM(email))
);
```

4.3 Idempotence et rejeu

Un batch de RDD doit toujours être **idempotent** : pouvoir être rejoué sans effet de bord. Les moyens d'y arriver :

- Contraintes UNIQUE sur clés métier + UPSERT dans les writers
- Flag `migre` sur la source (mis à jour en fin de batch)
- Table staging vidée/recréée au début de chaque exécution
- Spring Batch restart : `RunIdIncrementer` et reprise automatique

5. Ajouter un nouveau batch (pas à pas)

Voici la procédure complète pour ajouter une nouvelle reprise (ex : table source `COMMANDE_LEGACY` → tables cibles `COMMANDE`, `LIGNE_COMMANDE`, `LIVRAISON`).

Étape 1 — Documenter le mapping

Dans `docs/mapping-source-cible.md`, ajouter un tableau :

Colonne source	Table cible	Colonne cible	Règle
id_cmd	commande	id_externe	Clé de rapprochement
montant_ht	commande	montant_ht	Rejet si < 0
ref_prod_1..5	ligne_commande	reference_produit	1 ligne par ref non null
adresse_livraison	livraison	adresse	Normalisation

Ce document est validé par la MOA avant tout développement.

Étape 2 — Créer les entités source

Dans `domain/legacy/CommandeLegacy.java` (lecture seule, `@Immutable`).

Étape 3 — Créer la table de staging

Migration SQL `V4__staging_commande.sql` :

```
CREATE TABLE stg_commande (
    id_cmd BIGINT PRIMARY KEY,
    -- colonnes métier copiées de la source
    -- colonnes techniques de pilotage :
    statut_traitement VARCHAR(20) NOT NULL DEFAULT 'NEW',
    motif_rejet VARCHAR(500),
    date_extraction TIMESTAMP NOT NULL DEFAULT NOW(),
    hash_ligne VARCHAR(64)
);
CREATE INDEX idx_stg_commande_statut ON stg_commande(statut_traitement);
```

Étape 4 — Créer les entités cibles

Dans `domain/cible/commande/`, avec cascades JPA :

```
@Entity
@Table(name = "commande")
public class Commande {
    @Id @GeneratedValue private Long id;
    @Column(unique = true) private Long idExterne;

    @OneToMany(mappedBy = "commande",
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<LigneCommande> lignes = new ArrayList<>();

    @OneToOne(mappedBy = "commande", cascade = CascadeType.ALL)
    private Livraison livraison;
}
```

Étape 5 — Créer le mapper

Dans `mapper/CommandeMapper.java`. C'est ici que vit la logique d'éclatement. Testable unitairement sans Spring.

```
@Component
public class CommandeMapper {

    public Commande toCommande(StgCommande stg) {
        Commande cmd = new Commande();
        cmd.setIdExterne(stg.getIdCmd());
        cmd.setMontantHt(stg.getMontantHt());

        // Éclatement des 5 colonnes produit en N lignes
        cmd.setLignes(extraireLignes(stg, cmd));

        // Livraison si adresse présente
        if (stg.getAdresseLivraison() != null) {
            cmd.setLivraison(construireLivraison(stg, cmd));
        }
        return cmd;
    }
    // ...
}
```

Étape 6 — Créer les steps

Créer dans `step/` :

- `step/extraction/ExtractionCommandeStepConfig.java` — source → staging
- `step/validation/ValidationCommandeTasklet.java` — règles métier en SQL
- `step/chargement/commande/ChargementCommandeStepConfig.java` — staging → cibles

Étape 7 — Ajouter les steps au Job

> **Ordre critique :** un `Commande` référence un `Client`, donc le chargement client doit être terminé avant le chargement commande.

```
@Bean
public Job repriseGlobaleJob(
        Step chargementClientStep,      // d'abord les parents
        Step extractionCommandeStep,
        Step validationCommandeStep,
        Step chargementCommandeStep,    // ensuite les enfants
        Step reconciliationStep) {

    return new JobBuilder("repriseGlobaleJob", jobRepository)
        .incrementer(new RunIdIncrementer())
        .start(chargementClientStep)
        .next(extractionCommandeStep)
        .next(validationCommandeStep)
        .next(chargementCommandeStep)
        .next(reconciliationStep)
        .build();
}
```

Étape 8 — Écrire les tests

- **Tests unitaires du mapper** (prioritaire) : cas nominaux, cas limites, cas de rejet
- **Tests d'intégration du step** avec Testcontainers (BDD jetable)
- **Test bout-en-bout du job** avec jeu de données représentatif

Étape 9 — Mettre à jour la doc exploitation

Dans `docs/runbook-exploitation.md`, ajouter :

- Comment relancer uniquement le chargement commande si KO
- Requête SQL pour voir les rejets commande
- Volume attendu et temps de traitement estimé

6. Bonnes pratiques et checklist

6.1 Bonnes pratiques transverses

Performance

- Chunk size entre 100 et 5000 selon le volume — mesurer, ne pas deviner
- Activer `hibernate.jdbc.batch_size` si JPA
- Désactiver les index secondaires pendant le chargement massif
- Paralléliser les steps indépendants (enfants entre eux)

Robustesse

- Toujours prévoir un `SkipListener` qui log les rejets en base ou fichier
- Utiliser `@StepScope` pour les beans qui dépendent du contexte d'exécution
- Prévoir un step de **réconciliation finale** qui compte source vs cible
- Alerter (mail, Slack) en cas de KO via un `JobExecutionListener`

Maintenabilité

- Organiser **par domaine métier** avant par couche technique
- La logique de mapping dans des classes dédiées, testables sans Spring Batch
- Documenter le mapping source↔cible dans un tableau Markdown versionné
- Écrire un **runbook d'exploitation** (comment relancer, où chercher les erreurs)

Idempotence

- Contraintes UNIQUE sur les clés métier (`id_externe`)
- UPSERT plutôt qu'INSERT dans les writers cibles
- Staging rejouable : TRUNCATE ou flag en début de batch

6.2 Checklist pour chaque nouveau batch

- ☐ Document de mapping source→cible créé et validé par la MOA
- ☐ Migration SQL pour la table de staging avec colonnes techniques
- ☐ Entité legacy dans `domain/legacy/` en lecture seule
- ☐ Entité staging dans `domain/staging/` avec statut de traitement
- ☐ Entités cibles dans `domain/cible/<domaine>/` avec cascades JPA
- ☐ Contrainte UNIQUE sur la clé métier (`id_externe`)
- ☐ Mapper créé dans `mapper/` avec tests unitaires
- ☐ Step d'extraction (source → staging)
- ☐ Step de validation (Tasklet SQL pour marquer KO)
- ☐ Step de déduplication si applicable
- ☐ Steps de chargement (un par table cible) dans le bon ordre FK
- ☐ SkipListener branché pour tracer les rejets
- ☐ Step de réconciliation finale
- ☐ Tests d'intégration avec Testcontainers
- ☐ Runbook d'exploitation mis à jour
- ☐ Volumétrie et temps de traitement mesurés en pré-prod

6.3 Erreurs fréquentes à éviter

Erreur	Conséquence	Solution
Lire/écrire direct source → cible sans staging	Rejeu impossible, données instables	Toujours passer par staging
Chunk size trop grand	OOM, rollback coûteux	Commencer à 500, mesurer
Pas de clé métier stable	Impossible de rejouer sans doublons	Ajouter `id_externe` UNIQUE
Ordre FK non respecté dans les steps	Échec d'insertion à cause des FK	Parent avant enfants
Pas de skip listener	Impossible de tracer les rejets	Ajouter log en base + CSV
Logique métier dans le Reader/Writer	Code non testable	Logique dans Mapper + Service
Pas de réconciliation finale	Écarts silencieux entre source et cible	Tasklet de comptage + seuil d'alerte

Mot de la fin : en reprise de données, ce qui coûte n'est pas le code mais les cas particuliers imprévus dans les données réelles. Prévoir du temps pour les itérations de recette avec la MOA, et bâtir une mécanique de rejeu fiable dès le départ.